

Universidad Técnica Federico Santa María

Departamento de Ingeniería Electrónica

Diseño FPGA con MATLAB

Informe de ejemplo



Autor Ejemplar

Profesor Guía

Valparaíso, 2 de marzo de 2026

Resumen

La utilización de descripciones de alto nivel para el prototipado rápido de algoritmos es una práctica recurrente tanto en la industria como en la academia. Esto se debe a que la abstracción y las herramientas disponibles facilitan el desarrollo, la depuración y la validación. Sin embargo, la mayoría de los prototipos debe implementarse en sistemas físicos embebidos que no cuentan con recursos para ejecutar directamente esas descripciones. En consecuencia, disponer de una herramienta que reduzca la brecha entre el modelado de alto nivel y la implementación embebida mejora los tiempos de desarrollo y la productividad, y además disminuye el error humano al sistematizar el proceso.

Esta memoria se enfoca en aportar a proyectos del ámbito académico. Para ello, explora la utilidad práctica de HDL Coder y HDL Verifier, herramientas de MATLAB para la generación automática de HDL desde scripts y diagramas de bloques dentro de un flujo de *High-Level Synthesis* (HLS). En particular, se propone y valida una metodología sistemática para usar e implementar el código generado por HDL Coder en distintos casos de estudio ejecutados en FPGA sobre diferentes plataformas y entornos de verificación. En uno de los casos, relevante para proyectos en curso en el Departamento de Electrónica, se evalúan tiempos de ejecución y resultados de verificación en cada plataforma. Los códigos fuente y la información necesaria para reproducir y extender los experimentos están disponibles en un repositorio de código.

A partir de los resultados experimentales, se concluye que HDL Coder, apoyado por HDL Verifier, es una herramienta útil para generar código HDL optimizado para FPGA y mantener equivalencia funcional con las descripciones originales en MATLAB.

Índice general

Resumen	1
1. Introducción	4
1.1. Motivación y contexto.....	4
1.2. Planteamiento del problema	5
1.3. Metodología general.....	6
1.4. Alcances y contribuciones	7
1.5. Organización del informe.....	8
2. Antecedentes	9
2.1. Síntesis de alto nivel y herramientas de generación de código en la industria ..	9
2.2. Herramientas de MATLAB para generación y verificación de código	10
2.2.1. Fixed-point designer.....	10
2.2.2. HDL Coder	10
2.2.3. HDL Verifier.....	11
2.2.4. Discusión	12
3. Metodología y desarrollo	13
3.1. Flujo de trabajo	13
3.2. Aplicaciones de ejemplo	14
3.2.1. Producto punto.....	14
3.2.2. PID con anti-windup	16
3.2.3. PI y contador para análisis de muestreo	16
3.2.4. SNN con y sin <i>stream</i>	17
3.2.5. ADMM/MPC como caso de estrés	17
3.3. Plataformas objetivo y configuraciones	18
3.4. Implementaciones	18
4. Resultados experimentales	19
4.1. Validación funcional de códigos generados.....	19
4.1.1. Producto punto (flujo base)	19
4.1.2. PID con anti-windup	20
4.1.3. Control PI y overclocking	20

4.1.4.	Contador y muestreo	21
4.1.5.	Núcleo de SNN	21
4.1.6.	Evaluación de ADMM en distintas plataformas	21
4.1.6.1	Caracterización tiempos de ejecución	22
4.2.	Métricas de implementación	23
5.	Conclusiones y trabajo futuro	24
5.1.	Conclusiones.....	24
5.2.	Trabajo futuro.....	25
6.	Anexos	26
6.1.	Material complementario.....	26
6.2.	Guía breve de reproducción	26
6.3.	Datos adicionales	27

Índice de figuras

3.1	Flujo de buenas prácticas para usar HDL Coder en la generación y validación de HDL.	15
-----	--	----

Índice de tablas

4.1	Resumen de mediciones por caso (completar con reportes de síntesis y <i>FIL</i>). .	23
-----	--	----

1. Introducción

El trabajo presentado en esta memoria busca evaluar la utilidad de las herramientas *HDL Coder* y *HDL Verifier* de MATLAB para generar y verificar código HDL a partir de descripciones de alto nivel, con énfasis en la implementación en *FPGAs* y la validación mediante *FPGA-in-the-Loop* (FIL). El foco se sitúa en documentar un flujo de trabajo reproducible que parta desde modelos en *MATLAB* y *Simulink*, continúe con la generación automática de código y concluya con la verificación funcional y temporal sobre hardware objetivo. Se busca complementar la documentación oficial de MATLAB con directrices prácticas y ejemplos que reflejen restricciones de memoria, temporización y arquitectura propias de sistemas embebidos y plataformas reconfigurables.

Se espera aportar evidencia experimental que permita a investigadores e ingenieros evaluar la validez del flujo de diseño propuesto y reducir la brecha entre la descripción de alto nivel y la implementación verificable en hardware. Esto habilita prototipado rápido con garantías funcionales y facilita la simulación con sistemas continuos más cercanos a escenarios reales. Todo el material generado (scripts, modelos, bitstreams y reportes) está organizado para que pueda reutilizarse o extenderse en futuros proyectos.

1.1. Motivación y contexto

Matlab es ampliamente utilizado en la academia y la industria para modelar, simular y analizar algoritmos. Sus entornos de programación (scripts *.m*, bloques de *Simulink*, *Stateflow*, entre otros) permiten trabajar con distintos niveles de abstracción y acceder a un vasto ecosistema de *toolboxes*. Sin embargo, la verificación funcional en un computador de escritorio no garantiza que el algoritmo pueda ejecutarse eficientemente ni de forma determinista en un sistema embebido o en una *FPGA*. Las abstracciones de alto nivel facilitan el desarrollo, pero ocultan detalles de recursos, latencia y temporización que resultan críticos cuando se apunta a implementaciones físicas.

Los lenguajes HDL como VHDL o Verilog exigen, en su escritura manual, considerar estructuras de hardware, sincronización, consumo de LUT, registros y bloques DSP, además de la gestión de *clock domains* y *reset*. *HDL Coder* promete traducir modelos de alto nivel a descripciones HDL sintetizables, mientras que *HDL Verifier* habilita la validación contra el modelo original, incluyendo co-simulación con simuladores de terceros y uso de *FIL*. Esta última técnica permite ejecutar el algoritmo directamente en *FPGA* y mantener el banco de pruebas en MATLAB/Simulink, lo que reduce el tiempo de depuración y acerca la validación a condiciones reales. Además, abre la puerta a probar bloques HDL existentes (de terceros) conectándolos al entorno de referencia para contrastar funcionalidad y temporización sin

reescribir el banco de pruebas.

El desafío práctico radica en que no todo modelo de alto nivel es directamente portable a hardware: tipos de datos, dimensiones dinámicas, estructuras de control, acceso a memoria y temporización deben adecuarse a las restricciones del dispositivo. Las capas de abstracción de Matlab no relevan al diseñador de entender estos aspectos, y una configuración inadecuada puede resultar en código HDL que no sintetiza, que no cumple con las restricciones de tiempo o que altera la funcionalidad. Además, el ciclo de iteración hardware–software (generar, sintetizar, cargar, medir) es costoso si no se cuenta con una metodología clara y automatizada.

En el trabajo desarrollado se observó que esta brecha no solo es técnica, sino también metodológica. Muchos errores aparecen por decisiones tomadas en etapas tempranas que parecen menores: un tipo de dato sin saturación explícita, una señal de habilitación omitida o una suposición incorrecta sobre la latencia de salida. Cuando estos detalles no se controlan de forma sistemática, el costo de depuración aumenta y la validación final se vuelve lenta e incierta. Por ello, además de evaluar herramientas, esta memoria enfatiza la necesidad de un proceso ordenado, con trazabilidad de configuraciones y criterios de validación repetibles.

Otro factor relevante es la heterogeneidad de los casos de estudio reales. En un mismo proyecto pueden coexistir bloques de control discreto, módulos puramente secuenciales escritos en HDL y algoritmos de cómputo intensivo con patrones de acceso a memoria no triviales. En ese escenario, la utilidad del flujo no depende únicamente de que funcione en un ejemplo aislado, sino de que permita integrar variantes con exigencias distintas de latencia, ancho de datos y frecuencia interna. Esta motivación orienta la selección de casos y el enfoque comparativo usado en los capítulos siguientes.

1.2. Planteamiento del problema

El objetivo central de esta memoria es diseñar, validar y documentar una metodología para llevar algoritmos descritos en MATLAB/Simulink a implementaciones verificadas en FPGA mediante *HDL Coder* y *HDL Verifier*, apoyándose en *FIL* para cerrar el ciclo de verificación sobre hardware. Además, se incorpora la evaluación de bloques HDL preexistentes (desarrollados fuera de MATLAB) y su verificación mediante *FIL* usando Simulink en un entorno más cercano a la realidad. Los objetivos específicos son:

- Definir un flujo que parta en la especificación de alto nivel y culmine en un bitstream funcional, destacando configuraciones clave de *HDL Coder* (tipos de dato fijos, gestión de recursos, *pipelining* y *resource sharing*).
- Validar la equivalencia funcional entre el modelo original y el código HDL generado mediante *HDL Verifier*, usando co-simulación y *FIL* para detectar divergencias numéricas o de temporización.

- Verificar HDL existentes en un entorno de *Simulink* que emula condiciones cercanas a un sistema real, conectando el bloque a modelos del proceso y usando *FIL*/co-simulación para confirmar su funcionalidad frente a estímulos y respuestas realistas.
- Documentar recomendaciones, limitaciones y advertencias que no aparecen de forma explícita en la documentación base, enfocadas en facilitar la extensión a nuevos algoritmos y plataformas.

Desde esta perspectiva, el problema puede formularse como una pregunta de ingeniería aplicada: ¿qué condiciones de modelado y verificación permiten que el HDL generado conserve funcionalidad, mantenga un costo de recursos razonable y cumpla la temporalidad requerida por el sistema? Responder esta pregunta exige evaluar el flujo completo, no solo la etapa de generación. Un diseño puede producirse sin errores sintácticos y aun así ser inviable por requerir sobre-reloj excesivo, por no alinearse con el muestreo del banco de pruebas o por introducir retardos incompatibles con el lazo de control.

En consecuencia, la memoria no se limita a reportar “casos exitosos”, sino que también integra escenarios donde fue necesario ajustar arquitectura, tipos de datos o estrategia de validación para alcanzar una implementación viable. Este enfoque permite que las conclusiones tengan valor práctico para proyectos futuros y evita reducir la discusión a resultados nominales de laboratorio.

1.3. Metodología general

El trabajo se estructura en cuatro etapas principales:

1. **Ajuste del modelo de alto nivel:** se preparan los algoritmos en Matlab/Simulink asegurando tipos fijos, dimensiones estáticas y estructuras de control sintetizables. Se aplican lineamientos de *modeling for HDL* (evitar recursión, matrices dinámicas, *while* sin cotas, etc.).
2. **Generación de código HDL:** utilizando *HDL Coder*, se producen descripciones en VHDL/Verilog. Se revisan los reportes de recursos y *critical paths*, y se aplican optimizaciones (p. ej., *pipeline* y repartición de recursos) para cumplir restricciones de área y frecuencia.
3. **Verificación funcional y temporal:** con *HDL Verifier*, se realiza co-simulación y se implementa *FPGA-in-the-Loop*. Esta fase comprueba equivalencia funcional respecto al modelo de referencia y detecta desviaciones introducidas por la cuantización o por la latencia de hardware.
4. **Documentación:** se miden tiempos de ejecución, utilización de recursos y *throughput*; se comparan con la ejecución en software y se crean guías prácticas en un repositorio Git para su reproducción [7].

Cada etapa está acompañada de scripts automatizados que permiten repetir el flujo con distintas variantes de los algoritmos, registrando configuraciones de *HDL Workflow Advisor*, archivos de constraints y bancos de pruebas usados en las pruebas FIL.

Como referencia de estructura metodológica y de organización del material, también se consideró el repositorio de Jared Soto orientado a *Embedded Coder* [6]. Aunque ese trabajo está enfocado en generación de código para microprocesadores (C/C++), su forma de documentar el proceso, ordenar evidencias y presentar resultados fue tomada como guía para este documento, adaptándola al contexto de generación y validación de HDL en FPGA.

Además, la metodología incorpora un criterio de iteración incremental. En lugar de aplicar optimizaciones agresivas desde el inicio, cada caso se desarrolla en tres niveles: versión funcional mínima, versión ajustada para sintetizabilidad y versión optimizada para recursos/temporización. Este orden permite aislar con mayor claridad el efecto de cada decisión y simplifica la depuración cuando aparecen diferencias entre el modelo y el HDL.

También se define un conjunto común de métricas para comparar casos heterogéneos: latencia reportada, sobre-reloj requerido, uso de operadores principales (multiplicadores, sumadores, registros, multiplexores y RAM), y comportamiento funcional en co-simulación/FIL. Con este marco, los resultados del capítulo experimental no dependen de percepciones cualitativas, sino de evidencia comparable entre ejemplos.

1.4. Alcances y contribuciones

El trabajo se acota a la exploración de *HDL Coder* y *HDL Verifier* para la generación y verificación de código HDL orientado a FPGAs. Se tomarán funciones y modelos en Matlab/Simulink que resuelven tareas concretas (p. ej., control, procesamiento de señales, optimización) y se adaptarán al flujo para ilustrar patrones de modelamiento, problemas frecuentes y estrategias de mitigación. Además, se incorporan ejemplos que usan HDL ya existente (IP de terceros) para mostrar cómo integrarlos y validarlos con *FIL* junto a los modelos de referencia.

Las principales contribuciones son:

- Evidencia empírica sobre la capacidad de *HDL Coder* para generar código sintetizable y eficiente a partir de modelos de alto nivel, destacando sus límites prácticos.
- Un flujo reproducible de verificación con *HDL Verifier* y *FIL* que detecta divergencias funcionales y de temporización antes de comprometer un diseño a producción.
- Un repositorio con ejemplos, bitstreams, scripts de automatización y guías breves que facilitan adoptar el flujo en nuevos proyectos, reduciendo tiempos de implementación y riesgo de errores de bajo nivel.

Adicionalmente, el trabajo aporta una lectura transversal del flujo, mostrando cómo decisiones de alto nivel se traducen en efectos concretos de implementación. Por ejemplo, se evidencia que la selección de punto fijo no debe entenderse solo como una restricción de herramienta, sino como una decisión de arquitectura que condiciona precisión, latencia y costo de hardware; asimismo, se muestra que el ajuste de *overclocking/oversampling* no es un parámetro accesorio, sino una condición necesaria para validar correctamente diseños multiciclo en *FIL*.

En términos de transferencia, la memoria busca dejar una base utilizable por estudiantes e investigadores que necesiten prototipar bloques HDL en plazos acotados, manteniendo un estándar mínimo de rigor experimental. Esto se refleja tanto en la estructura de capítulos como en el material complementario incorporado en anexos.

1.5. Organización del informe

El resto del documento complementa los entregables principales y se organiza como sigue:

- El Capítulo 2 resume herramientas de generación automática de código desde alto nivel, con énfasis en *HDL Coder* y *HDL Verifier*.
- El Capítulo 3 detalla el flujo propuesto, configuraciones críticas y bancos de prueba usados en co-simulación y *FIL*, junto con las plataformas FPGA evaluadas.
- El Capítulo 4 presenta los resultados experimentales: equivalencia funcional, utilización de recursos y tiempos de ejecución comparados.
- El Capítulo 5 expone conclusiones y pautas para trabajos futuros, incluyendo oportunidades de optimización y extensiones a otros algoritmos o dispositivos.

2. Antecedentes

Este capítulo ofrece un panorama general de herramientas de generación automática de código orientadas a hardware reconfigurable. Se destacan los enfoques de *High-Level Synthesis* (HLS) para traducir descripciones de alto nivel a HDL, su presencia en la industria y sus principales aplicaciones.

2.1. Síntesis de alto nivel y herramientas de generación de código en la industria

La síntesis de alto nivel (HLS) busca reducir la brecha entre la especificación algorítmica y la implementación física en FPGA, generando descripciones RTL (VHDL o Verilog) a partir de modelos con mayor nivel de abstracción. En este enfoque, la herramienta se encarga de tareas estructurales como planificación temporal, inserción de *pipeline*, reparto de operadores y ajuste de paralelismo. El diseñador, por su parte, mantiene el control de decisiones de arquitectura: precisión numérica, latencia objetivo, estrategia de memoria y límites de iteración [1, 3].

En plataformas reconfigurables actuales, HLS se utiliza tanto para prototipado rápido como para acelerar etapas de diseño donde la codificación RTL manual sería costosa en tiempo. Su valor práctico no está solo en “generar HDL”, sino en permitir ciclos iterativos de exploración: cambiar una restricción, regenerar, comparar recursos y validar nuevamente la funcionalidad. Esa dinámica es particularmente útil cuando se debe balancear desempeño y área en diseños con requerimientos estrictos de temporización.

En este contexto, el flujo moderno de desarrollo no termina en la síntesis. El valor real se obtiene al cerrar el ciclo con verificación funcional y temporal sobre hardware, por ejemplo mediante co-simulación y *FPGA-in-the-Loop*. Estas etapas permiten detectar inconsistencias que no son evidentes en simulación de alto nivel, como errores por cuantización, latencias desalineadas o efectos de saturación acumulada en lazo cerrado.

Finalmente, HLS también ha cambiado la forma de documentar diseños: además del código generado, se reportan métricas de trazabilidad (configuración usada, reportes de recursos, latencias, restricciones de reloj y casos de prueba), lo que facilita reproducibilidad y comparación entre variantes de un mismo algoritmo.

2.2. Herramientas de MATLAB para generación y verificación de código

El ecosistema de MathWorks para FPGA se apoya principalmente en tres componentes complementarios: modelado en MATLAB/Simulink como referencia funcional, ajuste numérico con *fixed-point*, y generación/verificación de HDL mediante *HDL Coder* y *HDL Verifier*. Este enfoque permite recorrer un flujo continuo desde el diseño algorítmico hasta la validación sobre hardware real, manteniendo una única fuente de verdad para comparar resultados.

Una ventaja concreta de este esquema es la trazabilidad entre decisiones de modelado y consecuencias físicas. Por ejemplo, modificar el número de bits fraccionarios o activar compartición de recursos no solo altera una simulación: también cambia latencia, utilización de operadores y exigencia de reloj interno en la implementación final. Esta relación directa entre modelo y reporte de síntesis es uno de los elementos más útiles para iterar con criterio de ingeniería.

2.2.1. Fixed-point designer

Antes de generar HDL, el modelo debe cumplir condiciones de sintetizabilidad: tamaños estáticos, lazos acotados, rutas de control deterministas y tipos de dato compatibles con hardware. En la práctica, esta etapa es donde más tiempo se invierte, porque define la robustez del flujo completo. Un modelo que “simula bien” en alto nivel puede fallar en síntesis si mantiene dinámicas ambiguas, conversiones implícitas o rangos numéricos mal acotados.

Para evitar esas fallas, se recomienda fijar explícitamente escalas, redondeo y saturación en bloques críticos, y validar la diferencia entre referencia y modelo cuantizado con bancos de prueba representativos. En este proyecto, esta práctica fue clave en ejemplos de control y optimización, donde pequeñas desviaciones numéricas pueden modificar de forma importante la respuesta en lazo cerrado.

2.2.2. HDL Coder

HDL Coder permite generar RTL (VHDL o Verilog) desde modelos de Simulink o funciones de MATLAB. El flujo se apoya en el *HDL Workflow Advisor*, que configura el tipo de objetivo, los relojes, el *oversampling* y las optimizaciones de arquitectura. Entre sus capacidades destacadas se incluyen: generación automática de *pipelines*, reparto de recursos en bucles (*resource sharing*), inserción de registros para cumplir temporización, y reportes de latencia y utilización. La herramienta exige que los modelos sean sintetizables: dimensiones estáticas, ciclos acotados y tipos de datos compatibles (preferentemente *fixed-point*) [4].

En la práctica, HDL Coder permite cerrar un ciclo iterativo de modelado, generación y revisión de resultados de síntesis. Este ciclo resulta más efectivo cuando se mantiene una

disciplina de configuración estable entre iteraciones, de modo que las diferencias observadas puedan atribuirse a cambios de arquitectura y no a cambios del procedimiento.

Un elemento crítico es la conversión de punto flotante a punto fijo. HDL Coder se integra con Fixed-Point Designer para proponer tamaños de palabra, evaluar error numérico y definir reglas de saturación y redondeo. Esta etapa determina el rango dinámico y la precisión, afectando directamente el uso de recursos y la equivalencia funcional. En consecuencia, la recomendación general es validar primero el modelo en punto fijo y luego generar HDL, evitando conversiones tardías que puedan introducir saturaciones no controladas [2].

Además, HDL Coder puede generar IP cores con interfaces estándar para integrarlos en flujos de diseño de FPGA. Este paso es relevante cuando el diseño se conecta a buses o a procesadores embebidos, ya que permite empaquetar el bloque como IP con interfaces configurables y documentación de puertos.

2.2.3. HDL Verifier

HDL Verifier complementa el flujo de HDL Coder con mecanismos de verificación. Permite realizar co-simulación entre Simulink y simuladores HDL de terceros, comparando la salida del modelo con la del HDL generado y reportando discrepancias. Esta etapa es especialmente útil para detectar errores de cuantización, latencias desalineadas o efectos de saturación que no aparecen en simulación puramente en software [5].

Una característica central es *FPGA-in-the-Loop* (FIL), donde el diseño HDL se sintetiza, se implementa en FPGA y se ejecuta en hardware real mientras Simulink actúa como banco de pruebas. El modo *lockstep* sincroniza ciclos de FPGA con pasos de simulación; el modo *free-running* permite que el hardware ejecute a su reloj interno y Simulink intercambie datos bajo demanda. El ajuste del *overclocking factor* es esencial para alinear el número de ciclos internos con el muestreo de Simulink, particularmente cuando el HDL tiene latencias multiciclo.

Otro parámetro relevante en FIL es el *output delay*. Este valor indica cuántos periodos de muestreo de entrada deben transcurrir antes de obtener una salida válida, independiente del reloj interno. En un sistema retroalimentado con tiempo de muestreo fijo, dicho retardo se traduce en una demora efectiva dentro del lazo. Por ejemplo, si el controlador fue diseñado para leer la planta cada 1 ms y actuar en ese mismo periodo, un *output delay* de 4 implica que el actuador solo recibe una acción cada 4 ms; la planta cambia cada 1 ms, pero el control solo se actualiza cada cuatro muestras. Esto equivale a introducir un retardo de 4 ms en el lazo, lo que reduce el margen de fase y puede degradar el desempeño o incluso inestabilizar el sistema. En esos casos, se debe reconsiderar la arquitectura (reducir latencia, ajustar el *overclocking*, o rediseñar el controlador para el nuevo periodo efectivo) antes de validar el comportamiento en hardware.

2.2.4. Discusión

Dentro del alcance de esta memoria, HDL Coder y HDL Verifier conforman un flujo cerrado: generación RTL, análisis de arquitectura, verificación funcional y validación temporal en FPGA. La experiencia práctica de los ejemplos confirma que la principal fuente de éxito o fracaso no es la herramienta en sí, sino la disciplina de modelado aplicada desde el inicio.

En términos metodológicos, se identifican cuatro criterios que se repiten en todos los casos exitosos: (i) definición temprana de tipos y escalas, (ii) control explícito de latencia mediante *pipeline/resource sharing*, (iii) coherencia entre muestreo del modelo y temporalidad interna del HDL, y (iv) validación incremental con bancos de prueba trazables. Cuando alguno de estos criterios se omite, la probabilidad de retrabajo aumenta de manera significativa.

Por esta razón, el aporte de este capítulo no es solo describir herramientas, sino establecer una base de diseño reproducible orientada a HDL, sobre la cual se construyen los resultados experimentales de los capítulos siguientes.

3. Metodología y desarrollo

Este capítulo describe el flujo de trabajo propuesto para usar *HDL Coder* y *HDL Verifier*, los casos de estudio seleccionados y la metodología de validación en simulación y *FIL*. El contenido complementa el material disponible en el repositorio de código, que concentra scripts, modelos y bitstreams necesarios para reproducir y extender los resultados.

3.1. Flujo de trabajo

El flujo considera: ajuste del modelo en Matlab/Simulink para tipos y estructuras sintetizables; configuración de *HDL Coder* para generación RTL; co-simulación y *FIL* con *HDL Verifier* para equivalencia funcional; y registro de configuraciones y reportes en el repositorio. Cada caso de estudio sigue la misma secuencia para facilitar la comparación entre algoritmos.

En términos operativos, la secuencia se implementa en seis pasos:

1. **Definición del DUT y banco de pruebas:** se selecciona el bloque o función objetivo y se construye un *testbench* reproducible, con entradas representativas y criterio de comparación explícito.
2. **Conversión y validación numérica:** cuando el diseño parte en flotante, se propone tipo fijo, se ajustan fracciones y se valida el error respecto al modelo de referencia.
3. **Generación HDL y análisis de reportes:** se ejecuta el *Workflow Advisor*, revisando latencia, requerimientos de reloj interno y reportes de recursos para decidir ajustes de optimización.
4. **Análisis de resultados de recursos, ciclos internos y ciclos de latencia:** se consolidan métricas de síntesis para comparar variantes de arquitectura y verificar coherencia entre costo de hardware y desempeño temporal.
5. **Verificación en co-simulación y FIL:** se contrasta la salida HDL frente al modelo original y se comprueba comportamiento temporal bajo la configuración de *overclocking/oversampling* correspondiente.
6. **Trazabilidad y empaquetado:** se registran configuraciones, bitstreams, scripts y resultados para permitir repetición del experimento o extensión a variantes del caso.

Este esquema se repite en todos los ejemplos porque permite comparar resultados con el mismo criterio metodológico y, al mismo tiempo, aislar con mayor claridad qué efecto produce cada decisión de modelado o síntesis.

Como criterio de control de calidad, cada iteración del flujo se cerró con una revisión mínima de cuatro puntos: integridad del testbench, coherencia de tipos en interfaces, consistencia temporal (latencia y sobre-reloj) y reporte de recursos. Si uno de estos puntos fallaba, la iteración no se consideraba válida para comparación en el capítulo de resultados. Esta regla simple evitó acumular variantes no comparables y mejoró la trazabilidad del proceso.

3.2. Aplicaciones de ejemplo

Los ejemplos seleccionados cubren control, verificación y cómputo intensivo para validar el flujo completo sin entrar en el detalle de cada implementación. Se incluyen: producto punto (flujo base y comparación fixed vs floating), PID con anti-windup (lazo cerrado y *FIL*), controlador PI para estudiar *overclocking*, contador para caracterizar muestreo, MPC/ADMM como caso exigente en recursos, y un núcleo de SNN para evaluar procesamiento por eventos. El objetivo de esta selección es obtener conclusiones comparables sobre equivalencia funcional, latencia y uso de recursos en distintos perfiles de diseño.

La selección no fue arbitraria: se buscó cubrir patrones de diseño complementarios. Producto punto representa un caso de datapath acotado, útil para observar de forma controlada el efecto de cuantización y compartición de operadores. PID y PI representan sistemas con realimentación, sensibles a retardo y saturación. El contador aporta una referencia temporal simple para estimar sobre-reloj requerido. SNN incorpora alto fan-in y empaquetado de señales, mientras ADMM/MPC tensiona el flujo en complejidad algorítmica y costo de recursos.

Con este conjunto, el flujo se evalúa en dimensiones distintas pero comparables: fidelidad funcional, robustez temporal y viabilidad de implementación. En otras palabras, cada ejemplo cumple un rol metodológico dentro del análisis global y no solo una función demostrativa aislada.

3.2.1. Producto punto

El caso de producto punto se utiliza como referencia base del flujo porque permite controlar con claridad la relación entre precisión numérica, latencia y utilización de recursos. El proceso parte con una función MATLAB simple y un banco de pruebas con múltiples entradas aleatorias para construir una referencia “golden”. A partir de esa referencia se ejecuta la conversión a punto fijo y la validación numérica, de modo que el diseño HDL se genere desde tipos y rangos coherentes con los datos de prueba.

En la etapa de generación se evaluaron dos configuraciones: con y sin *stream loops*. Con *stream loops*, el compilador comparte operadores aritméticos y exige un reloj interno más rápido, reduciendo recursos a costa de aumentar las restricciones temporales. Sin *stream loops*, el diseño queda más directo y combinacional, con menor complejidad de control pero peor escalabilidad cuando el tamaño del problema aumenta. El ejemplo también se ejecutó

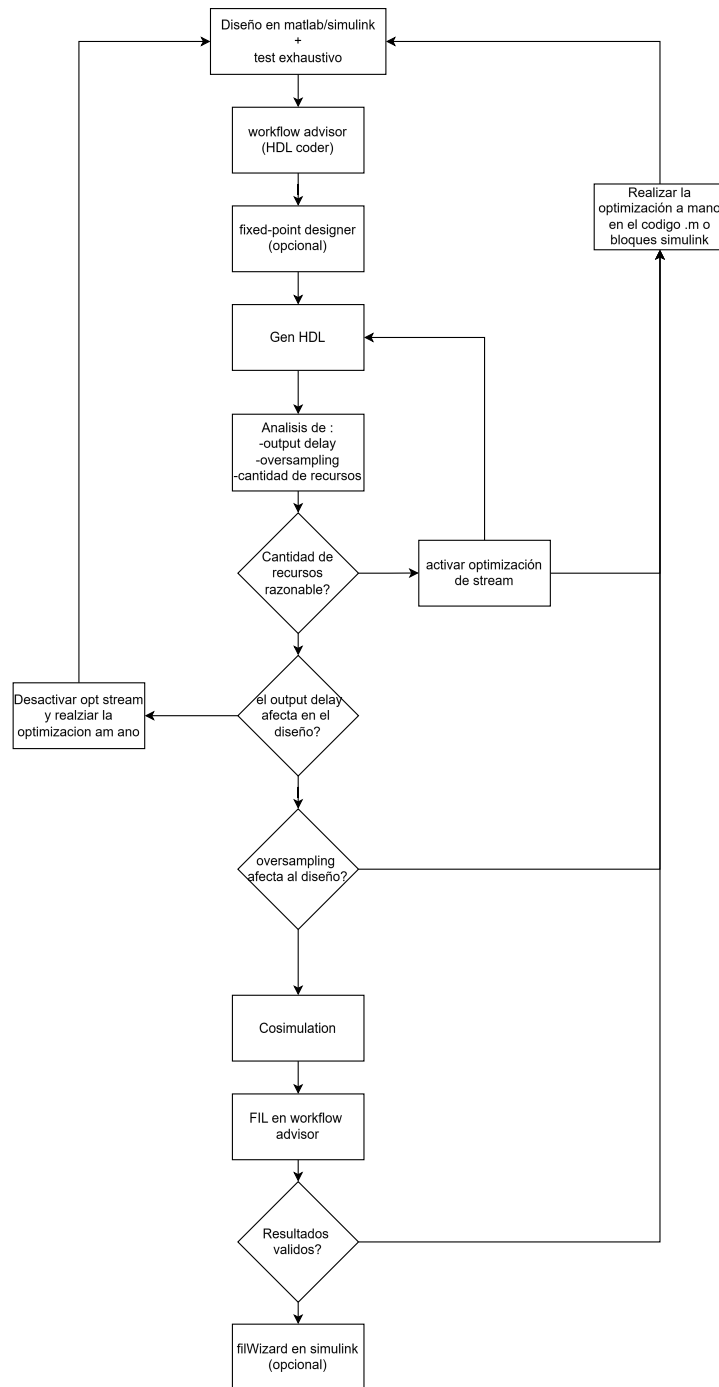


Figura 3.1: Flujo de buenas prácticas para usar HDL Coder en la generación y validación de HDL.

en punto flotante para contrastar costo en hardware frente a punto fijo, observándose un crecimiento importante de recursos para una mejora numérica que en este caso no era crítica.

Este caso también se utilizó para validar la disciplina de pruebas que luego se aplicó al resto de ejemplos: almacenamiento de señales de referencia, repetición de estímulos entre corridas y comparación explícita de salida esperada vs salida HDL. Al mantener fijo el conjunto de pruebas entre variantes, fue posible atribuir diferencias a cambios de arquitectura y no a cambios de estímulo.

3.2.2. PID con anti-windup

El ejemplo de PID con anti-windup se orienta a validar un flujo de control en lazo cerrado con parámetros configurables desde Simulink. El controlador se modela en tiempo discreto incorporando saturación de actuador y realimentación anti-windup en el integrador. Para la etapa HDL se fijan tipos *fixed-point* en entradas y bloques aritméticos, con saturación explícita en *overflow*, lo que reduce discrepancias entre el modelo de alto nivel y el comportamiento del HDL generado.

La verificación se realiza en dos niveles: primero por co-simulación y luego por *FPGA-in-the-Loop*. En ambas etapas se comparan señales del lazo cerrado para distintos escenarios de referencia y distintos valores de ganancia anti-windup. Esta secuencia permite observar, sobre un caso de control realista, cómo la elección de tipos y escalamiento impacta tanto la calidad de respuesta como la reproducibilidad funcional entre simulación y hardware.

En términos de implementación, este ejemplo fue clave para fijar lineamientos sobre señales de frontera entre dominios discretos y continuos. El uso de bloques de retención de orden cero y conversiones de tipo en puntos bien definidos evitó inconsistencias de muestreo y mejoró la interpretabilidad de resultados. Además, permitió analizar el efecto práctico de modificar parámetros del controlador sin regenerar por completo el entorno de validación.

3.2.3. PI y contador para análisis de muestreo

Los ejemplos de PI y contador se usan como casos instrumentales para estudiar el *overclocking factor*. En el PI, el comportamiento esperado se recupera cuando el *overclocking* coincide con los ciclos internos requeridos por el diseño; valores menores generan submuestreo y valores muy altos alteran la dinámica de la respuesta. En el contador, el efecto es aún más explícito: cada salida interna tiene una frecuencia distinta, por lo que el valor observado en Simulink depende de cuántos ciclos de FPGA se acumulan por cada muestra externa.

Ambos ejemplos se incluyeron porque entregan una regla práctica para configurar *FIL*: no basta con que el diseño sintetice, también debe existir coherencia entre la temporalidad interna del HDL y la temporalidad del banco de pruebas. Esta consideración se aplica luego al resto de casos, en particular a arquitecturas con latencia multiciclo.

Desde el punto de vista metodológico, PI y contador funcionaron como “instrumentos de

calibración” del flujo. Antes de validar casos más complejos, estos ejemplos permitieron comprobar que la infraestructura de pruebas (conexión FIL, sincronización y observación de señales) respondía de manera consistente frente a cambios controlados de *overclocking*. Esta calibración previa redujo incertidumbre al interpretar resultados de SNN y ADMM.

3.2.4. SNN con y sin *stream*

El núcleo SNN incorpora un patrón de cómputo por eventos y un ancho de entrada alto (784 señales binarias), lo que obliga a definir una estrategia de interfaz adecuada para *FIL*. En lugar de exponer cientos de puertos escalares, las entradas se empaquetan en un bus *ufix784*, permitiendo ejecutar pruebas por tramas sin perder trazabilidad con el modelo de referencia.

Para este caso se compararon dos variantes de síntesis: con *stream* y sin *stream*. La versión con *stream* prioriza compartición de recursos y mantiene un costo de hardware acotado; la versión sin *stream* favorece paralelismo estructural y aumenta de forma marcada el número de operadores. Esta comparación se incorporó para mostrar que, en diseños de gran fan-in, la política de compartición no es un ajuste secundario, sino una decisión de arquitectura que condiciona la factibilidad de implementación.

Adicionalmente, SNN permitió validar una estrategia de empaquetado de datos reutilizable en otros diseños con gran número de entradas binarias. Al transportar la información como bus compacto y mantener una convención consistente de orden de bits, se simplificó la integración con *FIL* y se redujo la complejidad de cableado del modelo de prueba.

3.2.5. ADMM/MPC como caso de estrés

El ejemplo ADMM/MPC representa el caso de mayor exigencia del conjunto. El algoritmo original, sin modificaciones para hardware, tiende a producir un RTL muy costoso en operadores y difícil de cerrar temporalmente. Por ello, el flujo incluye una fase previa de adecuación: acotar iteraciones, reordenar operaciones matriciales y privilegiar estructuras que faciliten *resource sharing*.

Este caso cumple dos funciones dentro de la metodología: validar que el flujo también opera sobre algoritmos de optimización más complejos y, al mismo tiempo, establecer límites prácticos del enfoque automático. En términos de documentación, ADMM/MPC actúa como referencia para discutir los compromisos entre precisión, latencia y recursos cuando se escala a aplicaciones de mayor complejidad.

La experiencia con ADMM/MPC también reforzó un criterio transversal de esta memoria: cuando un diseño no converge a una implementación razonable, la solución no suele ser “forzar más opciones” de la herramienta, sino rediseñar el cómputo para adaptarlo al modelo de hardware. Esta conclusión, aunque conocida en teoría, adquiere valor práctico al observarse de forma repetida en un caso real de alta exigencia.

3.3. Plataformas objetivo y configuraciones

Se emplean FPGAs de distintas gamas para validar funcionalidad y latencias: placas con dispositivos Artix/Cyclone para perfiles de recursos acotados y SoC reconfigurables (por ejemplo, Zynq) para evaluar integración con procesadores embebidos. Para *FIL* se definen relojes, interfaces de E/S y restricciones mínimas necesarias para ejecutar los bancos de pruebas sin optimizaciones específicas de plataforma.

La política de configuración priorizó portabilidad sobre ajuste fino. Es decir, se evitó sobre-optimizar cada caso para una única tarjeta y se privilegió una configuración estable que pudiera repetirse entre variantes. Esta decisión reduce el desempeño máximo alcanzable en algunos ejemplos, pero aumenta la comparabilidad de resultados y la utilidad del flujo como guía de trabajo general.

3.4. Implementaciones

Cada implementación incluye scripts de generación, archivos de proyecto y bancos de prueba. Los casos de control (PID y MPC) se verifican contra modelos de planta en *Simulink*, midiendo error de seguimiento y comportamiento bajo saturación. El núcleo SNN se valida con patrones de disparo conocidos y monitoreo de eventos para asegurar coincidencia con el modelo de referencia. Reportes de uso de recursos y frecuencia estimada se almacenan junto a los bitstreams generados.

Además, se mantuvo una convención común para documentar resultados: en cada ejemplo se reportan (i) configuración de tipos y optimizaciones, (ii) latencia observada o esperada, (iii) recursos principales de síntesis y (iv) comentario de equivalencia funcional en co-simulación/*FIL*. Esta forma de registro permite que el capítulo de resultados compare casos heterogéneos sin perder consistencia de lectura.

En la práctica, esta convención también permitió construir una línea de base para futuras extensiones. Cualquier nuevo caso que siga la misma estructura de documentación puede incorporarse al repositorio y compararse con los existentes sin redefinir métricas ni cambiar el formato de reporte. De esta manera, la memoria no solo describe un conjunto de experimentos, sino que deja establecido un método de trabajo escalable.

4. Resultados experimentales

Este capítulo consolida los resultados obtenidos al aplicar el flujo definido en capítulos previos. La presentación se organiza en dos niveles: primero se discute equivalencia funcional por tipo de caso; luego se comparan métricas estructurales de implementación (latencia y recursos). El objetivo no es reportar un récord de optimización, sino establecer evidencia consistente sobre qué decisiones de modelado y configuración permiten mantener funcionalidad y viabilidad de síntesis en FPGA.

4.1. Validación funcional de códigos generados

La validación funcional se realizó comparando el comportamiento del modelo de referencia en Matlab/Simulink con el HDL generado. Se trabajó en tres niveles: simulación a nivel de modelo, co-simulación HDL y ejecución *FPGA-in-the-Loop*. En cada etapa se aplicaron estímulos representativos y pruebas de borde para verificar saturación, truncamiento y reinicio. El foco estuvo en concluir si el flujo mantiene equivalencia funcional bajo las restricciones de hardware, sin detallar la implementación de cada ejemplo.

Para evitar diferencias por cuantización, los modelos se ajustaron a *fixed-point* con escalamiento explícito y saturación activada en bloques aritméticos críticos. La comparación se realizó en forma *bit-accurate* cuando fue posible y, en casos con punto flotante, se emplearon tolerancias acordes al error de cuantización. En términos cualitativos, la equivalencia funcional se observó en la respuesta transitoria y estacionaria; las discrepancias principales aparecieron en condiciones de saturación y en acumuladores integrales, lo que confirma la necesidad de ajustar el rango dinámico desde etapas tempranas y de usar bancos de prueba con señales representativas.

En todos los casos, la validación se consideró satisfactoria cuando se cumplieron simultáneamente tres criterios: (i) la forma de onda de salida mantuvo coherencia con la referencia, (ii) las diferencias numéricas estuvieron acotadas por el esquema de cuantización seleccionado, y (iii) la temporalidad observada en *FIL* fue consistente con la latencia reportada por la herramienta. Esta definición evita conclusiones parciales basadas solo en coincidencia visual o solo en métricas de síntesis.

4.1.1. Producto punto (flujo base)

La generación en punto fijo mostró buena correspondencia con el modelo de referencia dentro del rango de pruebas, con errores acotados por la precisión fraccional. El uso de *stream loops* permitió compartir multiplicadores en un producto punto 3x1, reduciendo recursos a costa

de mayor latencia interna y un requerimiento de reloj más alto. Cuando se desactiva el *streaming*, el HDL se vuelve combinacional, con menor latencia pero sin ahorro de recursos, lo que no escala en casos reales. En punto flotante se observó un aumento significativo de recursos y latencia sin beneficios claros para este ejemplo. La co-simulación y el *FIL* no mostraron errores para los datos testeados, lo que refuerza la necesidad de un banco de pruebas representativo y de rangos bien definidos.

Desde el punto de vista comparativo, este caso muestra una tendencia que se repite en diseños más grandes: para operaciones aritméticas simples, punto fijo ofrece una relación más favorable entre precisión y costo de hardware. El valor del ejemplo no está en su complejidad, sino en que permite observar de forma controlada cómo cambia la arquitectura al activar compartición de recursos y cómo ese cambio se refleja en latencia y sobre-reloj.

4.1.2. PID con anti-windup

El flujo *FIL* replica el comportamiento del controlador en lazo cerrado con parámetros configurables y evidencia que el anti-windup reduce el sobreimpulso frente a referencias altas. La equivalencia funcional depende de mantener coherencia de saturaciones, tiempos de muestreo y escalamiento entre Simulink y HDL. Al usar *fixed-point* con `fixdt` en entradas, sumadores y multiplicadores, y saturación en overflow, se evita que la salida exceda los límites físicos del actuador. Las pruebas con referencia elevada muestran que una ganancia de anti-windup más baja incrementa el sobreimpulso, mientras que una ganancia mayor reduce la saturación pero ralentiza la respuesta. El caso confirma que el control discreto es un candidato directo para generación automática si se cuidan los tipos de dato y la coherencia de lazo cerrado.

La comparación entre variantes fija y flotante en este caso aporta una conclusión práctica: aun cuando ambas preservan funcionalidad global, la variante en punto flotante incrementa de forma importante la complejidad estructural del diseño (en particular en registros y multiplexores), sin entregar una mejora proporcional en el objetivo de control bajo los escenarios de prueba considerados. Por ello, para esta clase de controlador, la configuración fija aparece como opción preferente cuando el objetivo es despliegue eficiente en FPGA.

4.1.3. Control PI y overclocking

El factor de *overclocking* debe coincidir con los ciclos internos del diseño para reproducir el comportamiento del modelo base. En el PI, la señal `D1_ap_vld` indicó que la salida válida ocurre cada 11 ciclos internos, por lo que un *overclocking* de 11 reproduce la respuesta del modelo. Un factor menor degrada la respuesta (submuestreo), mientras que un factor muy alto introduce sobreimpulso y mayor tiempo de asentamiento (sobremuestreo). Esto confirma que el ajuste de muestreo es crítico en la validación con *FIL* cuando existe cómputo multiciclo.

Este resultado es relevante porque muestra que un error de configuración temporal puede simular un “fallo de diseño” inexistente. En otras palabras, un controlador correctamente

implementado puede parecer inestable si se observa con una relación de muestreo inconsistente. La implicancia metodológica es directa: la interpretación de resultados en *FIL* siempre debe contextualizarse con latencia y sobre-reloj del DUT.

4.1.4. Contador y muestreo

El ejemplo del contador permite inferir el *overclocking* necesario a partir del periodo observado en la salida. El módulo implementa contadores a distintos ritmos (*cnt1*, *cnt10*, *cnt17*); al probar *OF* de 1, 10 y 17 se observa que cada canal reproduce el incremento esperado cuando el *OF* coincide con su frecuencia interna. Un *overclocking* alto produce acumulación interna y valores observados mayores a los esperados en los canales más rápidos, lo que muestra que el muestreo debe alinearse con la frecuencia efectiva de cada subcontador y que el *OF* puede estimarse a partir del periodo observado.

Aunque el caso es simple, su valor experimental es alto porque entrega una referencia cuantitativa clara para validar la infraestructura de *FIL*. Al trabajar con señales periódicas y expectativas directas de conteo, cualquier desalineación temporal se detecta de inmediato. Esta propiedad lo convierte en una prueba útil de diagnóstico antes de ejecutar validaciones sobre algoritmos más complejos.

4.1.5. Núcleo de SNN

La SNN muestra que el flujo es viable para arquitecturas por eventos y que la equivalencia se mantiene si el *oversampling* interno se configura de acuerdo con la latencia del diseño. Para evitar 784 puertos, las entradas se empaquetan en un bus *ufix784* y se procesan por frames. El log de HDL indica una latencia fija de 1 ciclo y un requerimiento de reloj interno de 7840 veces el *base rate*, por lo que el *oversampling* del bloque FIL debe coincidir con ese valor para replicar el comportamiento. La comparación flotante vs fijo evidenció diferencias menores que se corrigen ajustando bits fraccionarios. Se recomienda aumentar el número de frames para elevar la confianza estadística y cubrir casos borde.

La comparación entre síntesis con y sin *stream* mostró una diferencia estructural significativa: al desactivar compartición, el número de sumadores y multiplexores crece de forma abrupta, comprometiendo la escalabilidad del diseño. En cambio, la versión con *stream* mantiene un perfil de recursos más acotado y resulta más adecuada para despliegue real, aun cuando exige una configuración temporal más estricta en el entorno de verificación.

4.1.6. Evaluación de ADMM en distintas plataformas

El caso ADMM/MPC representa el límite práctico del flujo cuando la descripción original no está orientada a sintetizabilidad. Sin optimización, el uso de recursos es excesivo y la implementación en FPGA falla por tamaño. Al reestructurar los bucles y fijar el número máximo de iteraciones, se reduce la utilización y se obtiene un HDL viable, aunque con una

frecuencia interna elevada. La conclusión principal es que el algoritmo puede mantenerse funcional, pero requiere cambios explícitos en la forma de cómputo para que la herramienta explote el *resource sharing*.

En términos de interpretación, ADMM/MPC permite distinguir entre dos niveles de éxito. El primero es funcional: el algoritmo conserva comportamiento coherente respecto al modelo de referencia. El segundo es arquitectónico: el diseño logra una ocupación de recursos compatible con la FPGA objetivo y una latencia utilizable dentro del sistema. La experiencia del proyecto indica que alcanzar el primer nivel no garantiza automáticamente el segundo, especialmente en algoritmos iterativos con alta densidad de operaciones matriciales.

Los errores numéricos se concentran en cuantización de coeficientes y truncamiento intermedio. En ADMM, la cuantización de matrices impacta la solución óptima y puede introducir oscilaciones leves si la escala no es adecuada. El uso de punto fijo exige validar rangos máximos para evitar saturaciones silenciosas, especialmente cuando se comparten recursos y se incrementa la latencia interna.

Para mitigar estos efectos se recomienda aplicar escalamiento por etapa, incluir márgenes de saturación y validar con señales que cubran la dinámica completa. Con tamaños de palabra adecuados se observan diferencias acotadas y consistentes entre co-simulación y *FIL*.

Un aspecto relevante es que el error numérico no se distribuye de manera uniforme: suele concentrarse en etapas donde convergen acumulaciones o donde la dinámica del algoritmo cambia de régimen. Por ello, la validación no debe limitarse a promedios globales; conviene inspeccionar segmentos críticos de la señal para detectar sesgos o saturaciones locales que puedan degradar el comportamiento del sistema completo.

4.1.6.1. Caracterización tiempos de ejecución

Los tiempos de ejecución se relacionan directamente con la latencia de pipeline y la frecuencia interna requerida. En ADMM, el costo está dominado por operaciones matriciales y por el número de iteraciones; al fijar el máximo de iteraciones se obtiene un tiempo determinista por muestra, con un *overclocking* alto. La implementación en FPGA entrega determinismo temporal, mientras que en software el tiempo depende del sistema operativo y de la carga.

En diseños validados con *FIL*, esta caracterización temporal debe interpretarse junto con el tipo de interfaz y la política de muestreo del banco de pruebas. Una latencia de hardware baja no necesariamente se traduce en mejor respuesta de sistema si el acoplamiento con Simulink introduce esperas adicionales o si el factor de sobre-reloj se configura por debajo del requerido. En consecuencia, la métrica temporal útil para comparación no es solo “ciclos del DUT”, sino “ciclos del DUT bajo configuración efectiva de prueba”.

Tabla 4.1: Resumen de mediciones por caso (completar con reportes de síntesis y *FIL*).

Caso	Latencia (ciclos)	MUTS	Adders	FF	Multiplexer	Frecuencia
PP	1	1	4	10	4	10 MHz
PID-AW	1	6	7	7	5	10 MHz
PID-AW-FP	1	6	79	949	1007	1 MHz
SNN	784	1	14	12	18	10 MHz
SNN-NS	1	0	7840	0	7850	100 kHz
ADMM	10	248	652	119	48	1 MHz

4.2. Métricas de implementación

Abreviaturas de casos: PP (Producto punto), PID-AW (PID anti-windup), PID-AW-FP (PID anti-windup en *floating-point*), SNN (núcleo SNN con *stream*), SNN-NS (SNN sin *stream*) y ADMM (ADMM/MPC).

La tabla resume una tendencia consistente a lo largo de los ejemplos: el costo de hardware crece rápidamente cuando se privilegia paralelismo explícito o aritmética de mayor complejidad, mientras que las estrategias de compartición y ajuste de tipos permiten mantener diseños viables en FPGA de recursos acotados. Este comportamiento refuerza el argumento central de la memoria: el rendimiento final no depende solo del algoritmo, sino de cómo se adapta su estructura al modelo de ejecución hardware y al flujo de verificación.

Desde una perspectiva metodológica, *HDL Coder* demuestra ser una herramienta de alto valor para acelerar el paso desde un modelo en Matlab/Simulink hacia una implementación ejecutable en FPGA. Su principal aporte no es “automatizar todo el diseño”, sino reducir la barrera de entrada y el tiempo de iteración en etapas tempranas: permite generar versiones funcionales en menos tiempo, comparar alternativas de arquitectura y validar hipótesis de manera más directa en co-simulación y *FIL*.

Sin embargo, los resultados también muestran que la herramienta no reemplaza por completo el trabajo de ingeniería digital. La calidad del HDL generado sigue dependiendo de decisiones de modelado, definición de tipos de dato, restricciones temporales y configuración del flujo. En ese sentido, *HDL Coder* se entiende mejor como un acelerador de productividad que como un sustituto del criterio de diseño RTL.

Para equipos o estudiantes con poca experiencia en HDL, este enfoque ofrece una ventaja concreta: habilita una ruta de aprendizaje progresiva, donde primero se obtiene una implementación funcional y luego se refinan aspectos de recursos, latencia y temporización. Así, la herramienta facilita prototipado rápido y validación temprana, manteniendo al mismo tiempo un marco práctico para evolucionar hacia diseños más robustos y eficientes.

5. Conclusiones y trabajo futuro

5.1. Conclusiones

Esta memoria confirma que el uso conjunto de *HDL Coder* y *HDL Verifier* permite establecer un proceso de desarrollo reproducible para pasar de una especificación algorítmica a una implementación en FPGA con evidencia experimental. La validación *FPGA-in-the-Loop* se consolidó como una etapa clave porque habilita pruebas con el mismo banco funcional mientras el procesamiento ocurre en hardware real, reduciendo la incertidumbre entre el comportamiento esperado y el observado en implementación.

Los resultados obtenidos en los casos de estudio muestran que la calidad del diseño final depende menos de la complejidad nominal del algoritmo y más de la disciplina de modelado aplicada desde el inicio. Cuando se definen formatos numéricos coherentes, rutas de datos claramente secuenciadas y límites explícitos de latencia, el flujo de generación se vuelve estable y la síntesis converge con menor retrabajo. En cambio, cuando la especificación numérica queda implícita o se posterga la validación temporal, aparecen desajustes que luego demandan más iteraciones de ajuste.

También se observó que la relación entre recursos y rendimiento no es lineal y debe interpretarse en contexto. Diseños con menor número de operadores pueden presentar latencias elevadas si el patrón de cómputo es secuencial, mientras que arquitecturas más paralelas elevan consumo de recursos, pero reducen la espera por muestra y mejoran la capacidad de operación en tiempo real. Esta tensión entre paralelismo, área y latencia atraviesa todos los ejemplos y refuerza la necesidad de establecer criterios de diseño antes de optimizar casos particulares.

Un aspecto relevante es que la automatización no sustituye el criterio de ingeniería; lo potencia. Las herramientas aceleran tareas repetitivas y mejoran la trazabilidad, pero la decisión sobre qué pipeline usar, qué precisión numérica adoptar y qué compromisos aceptar sigue siendo del diseñador. En ese sentido, la principal contribución práctica de este trabajo no es solo el conjunto de implementaciones obtenidas, sino el marco metodológico para replicarlas y extenderlas de forma controlada.

Finalmente, la evidencia reunida respalda que el enfoque adoptado es útil para docencia, prototipado y desarrollo aplicado. El flujo permite pasar desde ejemplos básicos hasta aplicaciones más exigentes sin cambiar de paradigma de trabajo, conservando continuidad entre modelado, verificación y despliegue. Esta continuidad facilita la formación de criterios técnicos sólidos y reduce la curva de adopción para nuevos proyectos de aceleración en FPGA.

5.2. Trabajo futuro

Como continuidad natural de este trabajo, se propone ampliar el conjunto de algoritmos evaluados con problemas de mayor dimensionalidad y con exigencias temporales estrictas. En particular, resulta pertinente incorporar variantes de control predictivo con horizontes extendidos, redes de inferencia con mayor profundidad y módulos de preprocesamiento orientados a señales ruidosas. Esta expansión permitiría caracterizar con más detalle el punto en que cada estrategia de paralelización deja de ser eficiente para una plataforma determinada.

Otra línea de mejora consiste en fortalecer la automatización del repositorio para transformar el flujo experimental en un entorno de evaluación continua. La integración de scripts de construcción, verificación y recolección de métricas permitiría regenerar tablas y figuras de manera sistemática cada vez que se modifica un modelo. Con ello, la comparación entre versiones pasaría a estar respaldada por evidencia homogénea y no por campañas manuales de prueba.

También se plantea profundizar en métodos de refinamiento numérico orientados a minimizar error sin penalizar área. El análisis de sensibilidad por bloque, la selección dinámica de escalas y la validación cruzada entre precisión y latencia pueden aportar reglas más precisas para decidir formatos *fixed-point*. Este enfoque sería especialmente valioso en algoritmos iterativos, donde pequeños errores de cuantización se acumulan y afectan la convergencia global.

Desde la perspectiva de plataforma, es conveniente evaluar familias de FPGA con diferentes balances entre lógica programable, memoria interna y capacidad de DSP. Una campaña comparativa con criterios comunes de frecuencia, latencia y ocupación permitiría derivar guías de selección de hardware más concretas para proyectos futuros. Asimismo, la exploración de arquitecturas heterogéneas puede abrir nuevas posibilidades cuando conviene distribuir tareas entre procesamiento general y aceleración dedicada.

Por último, se propone extender la documentación metodológica con protocolos de validación por niveles, desde pruebas unitarias de bloques hasta evaluación integrada en lazo cerrado. Este tipo de documentación no solo incrementa la reproducibilidad, sino que facilita la transferencia del flujo a otros equipos de trabajo. Con ello, la memoria deja de ser una fotografía puntual de resultados y se convierte en una base técnica para desarrollos posteriores.

6. Anexos

6.1. Material complementario

El repositorio asociado a esta memoria concentra los modelos, scripts y reportes necesarios para reproducir cada resultado presentado en los capítulos anteriores. Su organización fue pensada para que un lector pueda repetir el flujo completo sin depender de configuraciones ocultas ni de pasos manuales ambiguos. Por esta razón, cada ejemplo mantiene una estructura estable, con artefactos de entrada, configuraciones de generación HDL y evidencias de salida separadas por caso de estudio.

En particular, se incluyen:

- Modelos de cada caso de estudio, con variantes en *fixed-point* y configuraciones orientadas a implementación en FPGA.
- Scripts para ejecución del flujo, generación HDL, preparación de pruebas y configuración de *FIL*.
- Archivos de proyecto y restricciones de temporización para las plataformas objetivo.
- Reportes de síntesis y utilización de recursos, junto con bitstreams generados.

El objetivo del material complementario es permitir que el lector replique el flujo completo y evalúe variaciones del diseño sin reestructurar los modelos base. Esta capacidad de re-utilización es relevante porque permite iterar sobre un mismo ejemplo cambiando solo una variable de interés, por ejemplo precisión numérica, nivel de *pipeline* o estrategia de recursos, y observar su impacto directo en las métricas finales.

6.2. Guía breve de reproducción

Para facilitar la repetición de resultados, se propone seguir una secuencia uniforme para todos los ejemplos:

1. Verificar que el modelo del caso de estudio ejecute correctamente en simulación funcional con los estímulos de prueba definidos.
2. Ejecutar la generación HDL y revisar mensajes de consistencia sobre tipos, latencias e inferencia de bloques.

3. Correr síntesis en la plataforma objetivo y registrar ocupación de multiplicadores, sumadores, registros, multiplexores y memorias.
4. Activar validación *FIL* para comparar salidas de hardware contra referencia funcional bajo los mismos estímulos.
5. Consolidar resultados en tablas comparables entre ejemplos, manteniendo criterios unificados de reporte.

Esta secuencia evita que la validación quede reducida a una sola evidencia y promueve una lectura integral del diseño: corrección funcional, viabilidad de síntesis y comportamiento temporal en hardware. Además, la repetición disciplinada de los mismos pasos entre casos ayuda a detectar con rapidez cuándo una diferencia de resultados se debe al algoritmo y no a cambios en el procedimiento.

6.3. Datos adicionales

Para facilitar la comparación entre versiones, se mantuvieron registros de configuración del *HDL Workflow Advisor*, versiones de entorno y parámetros de hardware. Adicionalmente, se documentan las señales de prueba utilizadas en simulación y los criterios de aceptación para equivalencia funcional en la validación *FIL*.

Como apoyo a la interpretación de resultados, se incorporan también observaciones cualitativas de cada corrida: advertencias de síntesis, ajustes requeridos para cierre temporal y decisiones de arquitectura aplicadas durante la iteración. Este tipo de anotaciones complementa las tablas numéricas, ya que explica por qué dos versiones con métricas similares pueden tener costos de implementación muy distintos en la práctica.

Se recomienda mantener la trazabilidad de cambios en el repositorio y actualizar las tablas cuando se incorporen nuevos algoritmos o plataformas. Esta práctica permite construir una base de evidencia acumulativa y comparar de forma consistente el impacto de nuevas configuraciones, mejoras del modelo o refinamientos numéricos. En conjunto, los anexos no solo respaldan los resultados obtenidos, sino que definen una ruta concreta para extender el trabajo en futuras iteraciones.

Bibliografía

- [1] Philippe Coussy y Adam Morawiec. *High-Level Synthesis: From Algorithm to Digital Circuit*. Springer, 2009. URL: <https://link.springer.com/book/10.1007/978-1-4020-8588-8>.
- [2] *Fixed-Point Designer*. MathWorks. 2024. URL: <https://www.mathworks.com/products/fixed-point-designer.html>.
- [3] Olivier Gaide y Philippe Coussy. «High-level synthesis: A decade of progress». En: *Design Automation Conference (DAC)*. 2011. URL: <https://doi.org/10.1145/2024724.2024734>.
- [4] *HDL Coder*. MathWorks. 2024. URL: <https://www.mathworks.com/products/hdl-coder.html>.
- [5] *HDL Verifier*. MathWorks. 2024. URL: <https://www.mathworks.com/products/hdl-verifier.html>.
- [6] Jared677. *Tutorial Embedded Coder repository*. https://gitlab.com/Jared677/embedded-coder/-/tree/main/Tutorial/Embedded_Coder?ref_type=heads. Consultado: 9 de febrero de 2026. 2026.
- [7] llillo2. *PID anti-windup example repository*. https://github.com/llillo2/memoria_fpga_design_matlab/tree/main/examples/PIDaw. Consultado: 9 de febrero de 2026. 2026.